As with class methods, you call a module method by preceding its name with the module's name and a period. As a result, one method is now accessible as Trig.sin and the other is Moral.sin and the names no longer conflict. Module constants are referenced using the module name followed by two colons, which is called the *scope resolution* operator, so in this example, Trig::PI and Moral::VERY_BAD.

Mixins

Modules have another, wonderful use. They can provide an alternative to inheritance as way of extending classes. This facility is sometimes called a *mixin*. Mixins enable something very much like multiple inheritance in other languages.

In the previous section's examples, we defined module methods, methods whose names were prefixed with self. If this made you think of class methods, your next thought may well be "What happens if I define instance methods within a module?" Good question. A module can't have instances, because a module isn't a class. However, you can *include* a module within a class definition. When this happens, all the module's instance methods are suddenly available as instance methods in the class as well. They get *mixed in*. In fact, for method lookup, mixed-in modules effectively behave as superclasses.

tut_modules/who_am_i.rb

```
module Debug
  def who_am_i?
    "#{self.class.name} (id: #{self.object_id}): #{self.name}"
  end
end

class Phonograph
  include Debug

  attr_reader :name

  def initialize(name)
    @name = name
  end
  # ...
end

class EightTrack
  include Debug

  attr_reader :name

  def initialize(name)
    @name = name
  end
  # ...
end

phonograph = Phonograph.new("West End Blues")
eight_track = EightTrack.new("Surrealistic Pillow")

phonograph.who_am_i?   # => "Phonograph (id: 60): West End Blues"
eight_track.who_am_i?  # => "EightTrack (id: 80): Surrealistic Pillow"
```

By including the Debug module, both the Phonograph and EightTrack classes gain access to the who_am_i? instance method.